The C Programming Language

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

Introduction

- **C++ features not in C**: This is clearly a non-exhaustive list, just the less obvious ones:

    1. Classes
    2. Member functions
    3. Constructors
    4. Destructors
    5. Access control: public, private, protected
    6. Inheritance
    7. Operator overloading
    8. Function overloading
    9. constexpr functions
    10. consteval
    11. constinit
    12. Namespaces
    13. References
    14. Move semantics
    15. RAII as a language-supported idiom
    16. Exceptions
    17. try / catch / throw
    18. new / delete
    19. Iterators
    20. Lambdas
    21. Capturing lambdas
    22. auto type deduction in the C++ sense
    23. decltype
    24. typeid
    25. dynamic_cast, static_cast, reinterpret_cast, const_cast
    26. Default function arguments
    27. Default member initializers
    28. Uniform initialization: initialization
    29. Member initializer lists
    30. this pointer
    31. Scoped enums: enum class
    32. bool as a built-in fundamental type in older C comparison
    33. References to functions/objects
    34. Inline variables
    35. Overload resolution
    36. Member pointers

Input / output

- **stdio.h**: In C, input/output is mostly handled through the standard I/O library, declared in:

```
#include <std.io>
```

- **Standard streams**: When a C program starts, it automatically has three standard streams:

```
stdin
stdout
stderr
```

- **printf**: Printf prints formatted text to stdout.

```
printf("%s %s", "Hello, ", "world!");
```

  The placeholders are called format specifies. Common ones include

```
%d      int
%u      unsigned int
%ld     long
%lu     unsigned long
%lld    long long
%llu    unsigned long long

%f      double
%lf     double in scanf, but also accepted in printf
%c      char
%s      string / char array
%p      pointer address
%%      literal percent sign
```

  For %p, cast the pointer to void*. Note that the language itself does not make printf typesafe. Wrong format specifiers cause **undefined behavior**.

- **fprintf**: The fprintf function in C is used to write formatted data directly to an output stream, which is most commonly a file. It works identically to printf, except it requires a destination file pointer as its very first argument.

```
int fprintf(FILE *stream, const char *format, ...);
```

- **sprintf**: The sprintf function in C formats and stores a series of characters and values into a string buffer instead of printing them directly to the console. It is defined in the <stdio.h> standard library.

```
int sprintf(char *str, const char *format, ...);
```

Returns the total number of characters successfully written, excluding the null terminator (\0). If an error occurs, it returns a negative value.

- **snprintf**:

```
int snprintf ( char * s, size_t n, const char * format, ...
↪ );
```

Composes a string with the same text that would be printed if format was used on printf, but instead of being printed, the content is stored as a C string in the buffer pointed by s (taking n as the maximum buffer capacity to fill).

If the resulting string would be longer than n-1 characters, the remaining characters are discarded and not stored, but counted for the value returned by the function.

A terminating null character is automatically appended after the content written.

Returns the number of characters that would have been written if n had been sufficiently large, not counting the terminating null character. If an encoding error occurs, a negative number is returned.

The key rule is **size** is the total buffer size, including space for the null terminator.

```
char buf[10];
snprintf(buf, sizeof buf, "hello");
```

sizeof buf is 10, so snprintf may write at most 9 visible characters plus the final '\0'.

```
char buf[6];
snprintf(buf, sizeof buf, "hello");
```

This fits exactly. So, this is safe.

```
char buf[5];

int n = snprintf(buf, sizeof buf, "hello");
```

snprintf writes

```
'h' 'e' 'l' 'l' '\0'
```

in this case the return value is still 5, because ”hello” would have required 5 visible characters, not counting the null terminator. snprintf returns the number of characters that would have been written, excluding the null terminator.

```
char buf[5];

int n = snprintf(buf, sizeof buf, "hello");

if (n < 0) {
    // encoding or formatting error
} else if ((size_t)n >= sizeof buf) {
    // output was truncated
} else {
    // output fit
}
```

You can call snprintf once to find out how much space is needed:

```
int n = snprintf(NULL, 0, "value = %d", 123);
```

- **scanf**: scanf reads formatted input from stdin.

```
int x;
scanf("%d", &x)
```

Scanf needs the address because it has to modify the variable. Scanf returns the number of successful conversions. You should usually check the return value

```
int x;
if (scanf("%d", &x) != 1) {
    fprintf(stderr, "Invalid integer\n");
}
```

- **puts**: Writes a string to the console

```
puts("Hello, world!");
```

- **getchar**: reads one character from stdin

```
int c;
c = getchar();
```

Notice that getchar returns int, not char. That is because it may return EOF, which is a special value indicating end-of-file or error.

```c
int c;
while ((c=getchar()) != EOF) putchar(c);
```

- **putchar**: Writes one character to stdout

```c
putchar('A');
```

- **fgets for line input**: Avoid

```c
char name[32];
scanf("%s", name);
```

If the user types more than 31 characters, you can overflow the buffer. A safer approach is fgets:

```c
char name[32];

printf("Enter your name: ");

if (fgets(name, sizeof name, stdin) == NULL) {
    fprintf(stderr, "Input error\n");
    return 1;
}

printf("Hello, %s", name);
```

Fgets has signature

```c
char* fgets( char* str, int count, FILE* stream );
```

and returns **str** on success, null pointer on failure.

- **sscanf**: The sscanf function in C extracts and parses formatted data directly from a character string rather than reading from standard keyboard input (stdin). It is defined in the <stdio.h> library and stands for "String Scan Formatted".

```c
int sscanf(const char *str, const char *format, ...);
```

```c
const char *info = "ID:4023 Alice 28";
int id;
char name[20];
int age;

// Parse data by explicitly matching the literal "ID:" prefix
int items = sscanf(info, "ID:%d %19s %d", &id, name, &age);
```

– **%[set]:** A scanset. For instance, %[A-Za-z] reads only alphabetic characters.
– **%[∧set]:** An inverted scanset. For instance, %[∧,] reads everything up until it hits a comma.
– *** Modifier:** Suppresses assignment. For example, %*d matches and skips an entire integer without storing it.

2.1 Working with files

- **Header**: File I /O is mainly done through the same header

```
#include <stdio.h>
```

- **FILE type**: The central type is

```
FILE*
```

A FILE* is a handle to an open file stream. You do not manipulate the file directly. You open it, get a FILE *, use functions such as fgetc, fgets, fprintf, fread, and finally close it with fclose.

- **fopen**:

```
FILE *fopen(const char *restrict filename, const char
↪  *restrict mode);
```

Upon successful completion fopen(), fdopen(), and freopen() return a FILE pointer. Otherwise, NULL is returned and errno is set to indicate the error.

The mode string tells C whether you want to read, write, append, or work in binary mode.

Mode	Meaning
”r”	Open existing file for reading
”w”	Open file for writing; destroys old contents
”a”	Open file for appending
”r+”	Open existing file for reading and writing
”w+”	Open for reading and writing; destroys old contents
”a+”	Open for reading and appending
”rb”	Read binary
”wb”	Write binary
”ab”	Append binary

The primary difference between r+ and a+ modes in C file handling lies in how they handle file creation and where the data is written. While both allow you to read and write without erasing (truncating) the existing data, they behave very differently under the hood.

Feature	r+ (Read / Update)	a+ (Append / Update)
File Must Exist?	Yes (returns NULL if missing)	No (creates a new file if missing)
Initial Pointer Position	Beginning of the file	End of the file
Overwrites Content?	Yes (overwrites at current cursor)	No (always forces writes to EOF)
Effect of fseek() on Writes	Moves the cursor for the next write	Ignored (writes always go to the end)

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("data.txt", "r");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    fclose(fp);
    return 0;
}
```

- **fclose**:

```c
int fclose(FILE* stream);
```

fclose closes an open file stream. It also flushes any buffered output before closing the file.

You should call fclose on every file you successfully opened with fopen.

Upon successful completion, 0 is returned. Otherwise, EOF is returned and errno is set to indicate the error.

EOF is a negative integer constant defined in <stdio.h>.

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("output.txt", "w");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    fprintf(fp, "Hello file\n");

    if (fclose(fp) == EOF) {
        perror("fclose");
        return 1;
    }

    return 0;
}
```

- **The restrict keyword**: restrict is a type qualifier used on pointers to promise the compiler that the pointer is the only means to access its underlying memory object

  When you qualify a pointer with restrict, you guarantee that for the lifetime of that pointer, no other pointer or variable inside the same scope will read or write to that specific chunk of memory.

- **fprintf**:

```c
int fprintf(FILE *restrict stream, const char *restrict
↪ format, ...);
```

  fprintf works like printf, but it writes to a file stream instead of standard output.

  Returns the number of characters written on success.

  Returns a negative value on failure.

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("numbers.txt", "w");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    int x = 10;
    double y = 3.14;

    fprintf(fp, "x = %d\n", x);
    fprintf(fp, "y = %.2f\n", y);

    fclose(fp);
    return 0;
}
```

- **fputs**:

```c
int fputs(const char *restrict s, FILE *restrict stream);
```

fputs writes a string to a file. Unlike puts, it does not automatically add a newline.

Returns a non-negative value on success. Returns EOF on failure.

- **fputc**

```c
int fputc(int c, FILE *stream);
```

fputc writes a single character to a file. The character is passed as an int, but it usually represents a char.

Returns the character written on success. Returns EOF on failure.

- **fgetc**:

```c
int fgetc(FILE *stream);
```

fgetc reads one character from a file.

It returns an int, not a char, because it needs to represent every possible character plus the special value EOF.

Returns the character read on success. Returns EOF if the end of the file is reached or if an error occurs.

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("data.txt", "r");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    int ch;

    while ((ch = fgetc(fp)) != EOF) {
        putchar(ch);
    }

    fclose(fp);
    return 0;
}
```

- **fgets**:

```c
char *fgets(char *restrict s, int n, FILE *restrict stream);
```

fgets reads a line, or part of a line, from a file into a character buffer.

It reads at most $n - 1$ characters and then adds the null terminator '\0'.

If it sees a newline before the limit, it includes the newline in the buffer.

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("data.txt", "r");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    char line[256];

    while (fgets(line, sizeof line, fp) != NULL) {
        printf("%s", line);
    }

    fclose(fp);
    return 0;
}
```

This is the normal way to read a text file line by line in C.

- **fscanf**:

```c
int fscanf(FILE *restrict stream, const char *restrict
↪  format, ...);
```

fscanf reads formatted input from a file, similar to how scanf reads from standard input.

It is useful for simple structured input, such as integers, floats, and words.

However, fscanf can be fragile for complicated input. For many real programs, reading a full line with fgets and then parsing it manually is safer.

Returns the number of input items successfully matched and assigned.

Returns EOF if the end of the file is reached before any conversion happens, or if a read error occurs.

- **fputs**

2.2 Binary I/O

- **fwrite**

```
size_t fwrite(const void *restrict data_ptr, size_t
    ↪ element_size, size_t n_elements, FILE *restrict stream);
```

Writes raw binary data from memory into a file. To write one `int`:

```
fwrite(&x, sizeof x, 1, fp);
```

To write an array:

```
fwrite(arr, sizeof arr[0], count, fp)
```

Returns the number of characters written on success.

Returns a negative value on failure.

- **fread**:

```
size_t fread(void ptr[restrict size * n], size_t size,
    ↪ size_t n, FILE *restrict stream);
```

The function fread() reads n items of data, each size bytes long, from the stream pointed to by stream, storing them at the location given by ptr.

On success, fread() and fwrite() return the number of items read or written. This number equals the number of bytes transferred only when size is 1. If an error occurs, or the end of the file is reached, the return value is a short item count (or zero).

2.3 I/O helpers

2.3.1 Flushing

- **fflush**

```
int fflush(FILE* stream);
```

C file output is often buffered. That means data may sit in memory temporarily before being physically written to the file. fflush forces buffered output to be written immediately. It is mainly used for output streams.

Returns 0 on success. Returns EOF on failure.

2.3.2 File positioning

- **Intro**: Every open file stream has a current position. Reading or writing usually advances that position.

  You can move around in a file using fseek, ftell, and rewind.

- **fseek**:

```
int fseek(FILE* stream, long offset, int whence);
```

  fseek moves file position. Whence controls where the offset is measured from:

  - **SEEK_SET**: Beginning of file
  - **SEEK_CUR**: Current position
  - **SEEK_END**: End of file

  Returns zero on success, nonzero on failure.

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("data.txt", "r");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    if (fseek(fp, 0, SEEK_END) != 0) {
        perror("fseek");
        fclose(fp);
        return 1;
    }

    fclose(fp);
    return 0;
}
```

  Moves position to EOF

- **ftell**:

```
long ftell(FILE* stream);
```

  returns the current file position. For binary files, this is commonly used as a byte offset.

  Returns current position on success, -1L on failure.

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("data.txt", "r");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    fseek(fp, 0, SEEK_END);

    long size = ftell(fp);

    if (size == -1L) {
        perror("ftell");
        fclose(fp);
        return 1;
    }

    printf("File size: %ld bytes\n", size);

    fclose(fp);
    return 0;
}
```

- **rewind**:

```c
void rewind(FILE* stream);
```

reset the file position indicator in a stream

2.3.3 EOF and Errors

- **feof**:

```
int feof(FILE *stream);
```

The feof() function shall test the end-of-file indicator for the stream pointed to by stream.

The feof() function shall not change the setting of errno if stream is valid.

The feof() function shall return non-zero if and only if the end-of-file indicator is set for stream, otherwise returns zero.

- **ferror**

```
int ferror(FILE* stream);
```

ferror checks whether an error occurred on a file stream.

This is useful because some functions, such as fgetc, return EOF both for real end-of-file and for errors.

Returns nonzero if the error indicator is set, zero otherwise.

- **clearerr**

```
void clearerr(FILE* stream);
```

clearerr clears both the error indicator and the EOF indicator for a stream.

You usually do not need it in simple programs, but it matters if you want to continue using a stream after EOF or after handling an error.

- **perror**:

```
void perror(const char* s);
```

perror prints an error message describing the most recent library/system error.

It is commonly used after functions such as fopen, fread, fwrite, fflush, or fclose fail.

2.3.4 Removing and renaming files

- **remove**:

```
int remove(const char* filename);
```

  Remove deletes a file.

  Returns zero on success, nonzero on failure.

- **Rename**

```
int rename(const char* old, const char* new);
```

  rename changes the name or path of a file.

  It can also be used to move a file within the same filesystem.

  Returns 0 on success. Returns nonzero on failure.

Strings

- **String literals**: Consider

```
char* s1 = "Hello, world!";
const char* s2 = "Hello, world!";
```

These create read only string literals. You should always make string literals const. Attempting to modify an non-const string literal will cause a crash.

To make them mutable, attach them to a char array

```
char arr[] = "Hello, world!";
```

The compiler allocates a fresh block of writable memory on the stack (or global scope) and copies the contents of ”Hello” into it. This memory is completely safe to change.

- **string.h**: The main C header for working with strings. Contains the standard C functions for working with null-terminated strings and raw memory blocks.
- **strlen**

```
size_t strlen(const char* s);
```

Returns the length of a C string, not counting the null terminator.

```
char s[] = "Hello";

sizeof s // 6, includes \0
strlen(s) // 5, excludes \0
```

- **strcmp**:

```
int strcmp(const char *s1, const char *s2);
```

Compares two strings lexicographically.

A negative return value implies s1 is less than s2. A return value of zero implies s1 equals s2. A positive return value implies s1 is greater than s2.

Note: Strings **cannot** be compared with ==. Using this operator simply compares addresses.

Because strcmp does not accept a maximum length argument, it evaluates characters sequentially until it detects a null-terminator (\0). If a maliciously crafted or malformed string lacks a null-terminator,

  - Keep reading past the assigned memory buffer.
  - Trigger undefined program behavior.
  - Cause a segmentation fault (SIGSEGV) or application crash if it tries to read protected memory spaces.
  - Potentially read adjacent data from the stack or heap, giving attackers a mechanism to bleed sensitive information

- **strncmp**:

```
int strncmp(const char* s1, const char* s2, size_t n);
```

  Compares at most the first n characters of two strings.

  Has same return behavior as `strcmp`.

- **strcpy**:

```
char* strcpy(char* dest, const char* src);
```

  Copies the string src into dest, including the null terminator.

  Returns dest.

- **strncpy**

```
char *strncpy(char *dest, const char *src, size_t n);
```

  Copies at most n characters from src to dest.

  Returns dest.

  This function is **not** simply a safer strcpy. If src has length greater than or equal to n, strncpy does **not** append a null-terminator.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char dest[6];

    strncpy(dest, "hello world", sizeof dest - 1);
    dest[sizeof dest - 1] = '\0';

    printf("%s\n", dest);

    return 0;
}
```

- **strcat**:

```c
char *strcat(char *dest, const char *src);
```

Appends src to the end of dest. Starts at the null-terminator of dest. Dest must have a null-terminator and enough room for the result.

Returns dest.

- **strncat**:

```c
char *strncat(char *dest, const char *src, size_t n);
```

Appends at most n characters from src to dest, then appends a null-terminator.

Returns dest

- **strchr**:

```c
char* strchr(const char* s, int c);
```

Finds the first occurrence of character c is string s.

Returns pointer to first occurrence of character c in string s, or NULL if not found.

- **strrchr**

```c
char* strrchr(const char* s, int c);
```

Finds the last occurrence of character c is string s.

Same return value as strchr

- **strstr**

```
char* strstr(const char* haystack, const char* needle);
```

Finds the first occurrence of string **needle** inside string **haystack**.

Returns pointer on success, NULL if not found.

- **strtok**:

```
char* strtok(char* str, const char* delim);
```

Splits a string into tokens using delimiter characters.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "red,green,blue";

    char *token = strtok(s, ",");

    while (token != NULL) {
        printf("%s\n", token);
        token = strtok(NULL, ",");
    }

    return 0;
}
```

strtok modifies the original string by writing '\0' characters into it. Also notice that strtok uses a hidden internal static state. Essentially, you can pass NULL to it and it will pick up where it left off.

```
char s[] = "red,green,blue";
size_t n = sizeof s;

char *token = strtok(s, ",");

for (size_t i = 0; i < n; ++i) {
    printf("%c", s[i]);
    if (s[i] == '\0') {
        fputs("__null-term__", stdout);
    }
}

// red__null-term__green,blue__null-term__
```

```c
char s[] = "red,green,blue";
size_t n = sizeof s;

char *token = strtok(s, ",");

while (token != NULL) {
    token = strtok(NULL, ",");
}

for (size_t i = 0; i < n; ++i) {
    printf("%c", s[i]);
    if (s[i] == '\0') {
        // Using fputs instead of puts for no newline.
        fputs("\\0", stdout);
    }
}

// red\0green\0blue\0
```

- **strspn**:

```c
size_t strspn(const char* s, const char* accept);
```

Returns the length of the initial segment of s containing only characters from `accept`.

- **strcspn**:

```c
size_t strcspn(const char *s, const char *reject);
```

Returns the length of the initial segment of s containing no characters from reject.

- **strpbrk**:

```c
char *strpbrk(const char *s, const char *accept);
```

Finds the first character in s that matches any character in accept.

- **strerror**:

```c
char* strerror(int errnum);
```

Returns a human-readable error message for an error number.

Data structures

4.1 Linked list

- **Structures**:

```
typedef struct _Node {
    int data;
    struct _Node* next;
} Node;

void node_init(Node* node, int data) {
    node->data = data;
    node->next = NULL;
}

typedef struct _LL {
    Node* head, *tail;
} LL;
```

- **Appending**:

```
void append(LL* list, Node* node) {
    if (node == NULL) return;

    node->next = NULL;

    if (list->head == NULL) {
        list->head = node;
        list->tail = node;
        return;
    }

    list->tail->next = node;
    list->tail = node;
}
```

Its not guaranteed that

```
node->next == NULL
```

So, we ensure this ourselves.

- **Clearing**

```c
void clear(LL* list) {
    if (list->head == NULL) return;

    Node* curr = list->head;
    while (curr != NULL) {
        Node* forward = curr->next;
        curr->next = NULL;
        free(curr);
        curr = forward;
    }

    list->head = NULL;
    list->tail = NULL;
}
```

- **Traversing**:

```c
void ll_print(LL* list) {
    Node* curr = list->head;
    while (curr != NULL) {
        printf("%d, ", curr->data);
        curr=curr->next;
    }
}
```

- **Usage**:

```c
LL list = {0};

Node* n1 = malloc(sizeof(Node));
node_init(n1, 1);

Node* n2 = malloc(sizeof(Node));
node_init(n2, 2);

Node* n3 = malloc(sizeof(Node));
node_init(n3, 3);

Node* n4 = malloc(sizeof(Node));
node_init(n4, 4);

append(&list, n1);
append(&list, n2);
append(&list, n3);
append(&list, n4);

ll_print(&list);
puts("");

clear(&list);
ll_print(&list);
```

- **Finding**:

```
0  _Bool find(LL* list, Node* node) {
1      if (node == NULL  list == NULL) return false;
2
3      Node* curr = list->head;
4      while (curr != NULL) {
5          if (curr == node) return true;
6          curr=curr->next;
7      }
8      return false;
9  }
```

- **Erase**:

```
0  void erase(LL* list, Node** node) {
1      if (node == NULL  list == NULL) return;
2
3      if (!find(list, *node)) return;
4
5      // Single element
6      if (list->head == list->tail) {
7          free(list->head);
8          list->head = NULL;
9          list->tail = NULL;
10         *node = NULL;
11         return;
12     }
13
14     // If the node we want to remove is the head
15     if (list->head == *node) {
16         list->head = list->head->next;
17         free(*node);
18         *node = NULL;
19         return;
20     }
21
22     // General case
23     Node* curr = list->head;
24     Node* prev = list->head;
25     while (curr != NULL) {
26         if (curr == *node) {
27             curr = curr->next;
28             break;
29         }
30         prev = curr;
31         curr = curr->next;
32     }
33
34     // If the node we want to remove is the tail
35     if (*node == list->tail) {
36         list->tail = prev;
37     }
38
39     prev->next = curr;
40     free(*node);
41     *node = NULL;
42 }
```

Memory operations in string.h

- **memset**

```
void* memset(void* s, int c, size_t n);
```

Sets n bytes of memory to the byte value c.

Returns s

```
char buffer[8];

memset(buffer, 0, sizeof buffer);

printf("%d\n", buffer[0]);
```

- **memcpy**:

```
void *memcpy(void *dest, const void *src, size_t n);
```

Copies exactly n bytes from src to dest.

Returns dest.

```
int src[] = {1, 2, 3};
int dest[3];

memcpy(dest, src, sizeof src);
```

Overlapping memory is UB.

- **memmove**:

```
void *memmove(void *dest, const void *src, size_t n);
```

Copies exactly n bytes from src to dest, safely handling overlapping memory regions.

Returns dest.

- **memcmp**:

```
int memcmp(const void *s1, const void *s2, size_t n);
```

Compares exactly n bytes of memory.

Returns:

- < 0 first differing byte in s1 is less than the one in s2
- 0 all n bytes are equal
- > 0 first differing byte in s1 is greater than the one in s2

- **memchr**:

```
0  void* memchr(const void* s, int c, size_t n);
```

Searches the first n bytes of memory for byte value c.

Returns pointer to the first matching byte or NULL.

stdlib.h

6.1 Memory management

- **malloc**

```
0  void* malloc(size_t size);
```

Allocates `size` bytes of uninitialized memory.

Returns a pointer to the allocated memory, or NULL if allocation fails.

```
0  int* arr = malloc(10 * sizeof *arr);
```

The memory contains indeterminate values. Reading before writing is undefined behavior.

- **calloc**:

```
0  void* calloc(size_t n, size_t size);
```

Allocates space for n elements, each `size` bytes, and initializes all bytes to zero.

Returns a pointer to the allocated memory, or NULL if allocation fails.

```
0  int* arr = calloc(10, sizeof *arr);
```

For integer types, the values become 0. For pointer types, on normal modern systems this gives null pointers, but the standard technically describes byte zeroing.

- **realloc**:

```
0  void* realloc(void* ptr, size_t size);
```

Changes the size of a previously allocated block.

Returns a pointer to the resized block, or NULL if resizing fails.

```c
int *arr = malloc(5 * sizeof *arr);

int *tmp = realloc(arr, 10 * sizeof *arr);
if (!tmp) {
    free(arr);
    /* allocation failed */
} else {
    arr = tmp;
}

free(arr);
```

If you assign the return value of realloc to the original memory pointer, you loose it if realloc fails and returns NULL.

- **free**

```c
void free(void* ptr);
```

Releases memory allocated by malloc, calloc, or realloc.

Returns nothing.

```c
free(NULL); // safe
free(nullptr) // safe
```

Both of these are safe, but freeing the same pointer twice is undefined behavior.

6.2 String to number conversions

- **__Nullable**: Simply means that the pointer passed to the function is allowed to be NULL. It is however not a keyword in C (and doesn't occur even once in the draft for the C23 standard) but an extension supported only by some compilers, for example clang and icx.
- **atoi, atol, atoll**:

```
int atoi(const char* nptr);
long atol(const char* nptr);
long long atoll(const char* nptr);
```

Converts a string to int, long, or long long.

Returns the converted integer.

No reliable error reporting, prefer `strtol` or `strtoll`.

- **strtol, strtoll, strtoul, strtoull, strtod, strtof, strtold**

```
long strtol(const char *restrict nptr, char **restrict
↪  endptr, int base);
long long strtoll(const char *restrict nptr, char **restrict
↪  endptr, int base);
unsigned long strtoul(const char *restrict nptr, char
↪  **restrict endptr, int base);
unsigned long strtoull(const char *restrict nptr, char
↪  **restrict endptr, int base);
double strtod(const char *restrict nptr, char **restrict
↪  endptr);
float strtof(const char *restrict nptr, char **restrict
↪  endptr);
long double strtold(const char *restrict nptr, char
↪  **restrict endptr);
```

Returns the converted value. If endptr is not NULL, it is set to the first character after the parsed number.

```
char *end;
long x = strtol("123abc", &end, 10);

printf("%ld\n", x);   /* 123 */
printf("%s\n", end);  /* abc */
```

Base == 0 detects prefixes.

```
"123"   /* decimal */
"077"   /* octal */
"0xff"  /* hexadecimal */
```

```
char* end;
const char* s = "123a";
int x = strtol(s, &end, 10);

if (errno == ERANGE) {
    puts("Overflow or underflow");
}

else if (s == end) {
    puts("No conversion");
}

else if (*end == '\0') {
    puts("Full conversion");
}

else {
    puts("Some conversion");
}
```

6.3 Program termination and process control

- **exit**

```
void exit(int status);
```

  Terminates the program with exit code `status`.

  Returns nothing.

- **abort**:

```
void abort(void)
```

  Abnormally terminates the program.

- **atexit**:

```
int atexit(void (*func)(void));
```

  Registers a function to be called when the program exists normally.

  Returns zero on success, nonzero on failure.

  Functions registered with atexit are called in reverse order of registration.

- **Normal program exit cleanup**: A normal program exit (triggered by returning from main() or calling exit()) automatically performs standard runtime and stream cleanup, while application-level resource cleanup remains the responsibility of the code.

  When a program terminates normally, the C standard library (stdlib.h) executes the following operations automatically in order:

  - **Calls Termination Functions:** Executes functions registered via the atexit() function in the reverse order of their registration.
  - **Flushes and Closes Streams:** Flushes all unwritten buffered data to open C streams (like stdout or stderr) and closes all open files.
  - **Deletes Temporary Files:** Removes any temporary files created using tmpfile().
  - **Returns Control:** Passes the final exit status code back to the host environment (e.g., to the operating system or parent process).

- **_Exit**:

```
void _Exit(int status);
```

Terminates the program immediately. No `atexit` handlers are called and normal cleanup is not performed.

- **system**

```
int system(const char* string);
```

Passes a command string to the host environment's command processor.

Returns implementation-defined status information.

You can also check whether a command processor exists:

```
if (system(NULL)) {
    puts("command processor available");
}
```

The return value is

  - If command is NULL, then a nonzero value if a shell is available, or 0 if no shell is available.
  - If a child process could not be created, or its status could not be retrieved, the return value is -1 and errno is set to indicate the error.
  - If a shell could not be executed in the child process, then the return value is as though the child shell terminated by calling _exit(2) with the status 127.
  - If all system calls succeed, then the return value is the termination status of the child shell used to execute command. (The termination status of a shell is the termination status of the last command it executes.)

- **getenv**:

```
char* getenv(const char* name);
```

Gets the value of an environment variable.

Returns a pointer to the value string, or NULL if the variable does not exist.

Do not modify the returned string unless your platform specifically allows it. Treat it as read-only.

6.4 Random numbers

- **rand**

```
int rand(void);
```

Returns a pseudo-random integer in the range 0 to RAND_MAX.

- **srand**

```
void srand(unsigned int seed);
```

seeds the pseudo-random number generator used by rand.

If you do not call `srand`, the program behaves as though it were seeded with 1.

6.5 Sorting and searching

- **qsort**

```
void qsort(void *base, size_t nmemb, size_t size, int
↪  (*compare)(const void *, const void *));
```

Sorts and array. Returns nothing

The comparison function must return:

- < 0 if first element comes before second
- 0 if elements compare equal
- > 0 if first element comes after second

```
#include <stdio.h>
#include <stdlib.h>

int cmp_int(const void *a, const void *b) {
    const int *x = a;
    const int *y = b;

    if (*x < *y) return -1;
    if (*x > *y) return 1;
    return 0;
}

int main(void) {
    int arr[] = {5, 2, 9, 1, 3};
    size_t n = sizeof arr / sizeof arr[0];

    qsort(arr, n, sizeof arr[0], cmp_int);

    for (size_t i = 0; i < n; ++i) {
        printf("%d ", arr[i]);
    }
}
```

- **bsearch**:

```
void *bsearch(const void *key, const void *base, size_t
↪  nmemb, size_t size, int (*compar)(const void *, const
↪  void *));
```

Performs binary search on a sorted array.

Returns pointer to the matching element, or NULL.

```c
#include <stdio.h>
#include <stdlib.h>

int cmp_int(const void *a, const void *b) {
    const int *x = a;
    const int *y = b;

    if (*x < *y) return -1;
    if (*x > *y) return 1;
    return 0;
}

int main(void) {
    int arr[] = {1, 2, 3, 5, 9};
    int key = 5;

    int *found = bsearch(&key, arr, 5, sizeof arr[0],
    ↪  cmp_int);

    if (found) {
        printf("found %d\n", *found);
    }
}
```

6.6 Integer math helpers

- ```
  int abs(int j);
  long labs(long j);
  long long llabs(long long j);

  div_t div(int numer, int denom);
  ldiv_t ldiv(long numer, long denom);
  lldiv_t lldiv(long long numer, long long denom);
  ```

- **div_t**:

  ```
  typedef struct {
      int quot;
      int rem;
  } div_t;
  ```

  Its variants are similar.
  ```

Concepts

- **Tag names**: In C, a tag name is the name that belongs to a struct, union, or enum declaration. For example,

```
struct Node {
    int value;
};
```

Here, `Node` is the **tag name**. The actual type is `struct Node`. So, to declare a variable

```
struct Node n;
```

This is why we see

```
typedef struct Node {...} node;
typedef union _U {...} U;
```

Then,

```
U u;
Node n;
```

Are both legal. If we don't care about writing

```
Union _U u;
Struct Node n;
```

We can omit the tag name,

```
typedef struct {...} Node;
typedef union {...} U;
```

Then, only

```
Node n;
U u;
```

Are legal.

- **Functions that return function pointers**: Consider a function f that accepts a single integer.

```
f(int x) {...}
```

Suppose f returns a pointer a function that returns void and accepts a single double.

```
typedef void (*fp)(double);
fp f(int x) {...}
```

But, what if we didn't want to use typedef. Then, we wrap the function f inside the function pointer type.

```
void (*f(int x))(double y) { ... }
```

Notice how fp became the signature (excluding a return value) of f. Read as: "f" is a function that takes an int and returns a pointer to a function that takes a double and returns void.

- **Unions**: C code uses union more often because C has fewer built-in abstraction tools. In C++, many uses of union are replaced by safer language/library features such as classes, inheritance, templates, std::variant, std::optional, and RAII-managed types.

A union is useful when you want several different members to occupy the same memory location, but only one of them is valid at a time

```
union Value {
    int i;
    double d;
    char* s;
}
```

The size of Value is large enough to hold the largest member. The fields overlap in memory.

A classic C pattern is a tagged union:

```c
enum ValueKind {
    VALUE_INT,
    VALUE_DOUBLE,
    VALUE_STRING
};

struct Value {
    enum ValueKind kind;

    union {
        int i;
        double d;
        char* s;
    } as;
};
```

The kind field tells you which union member is currently valid.

- **SIZE_MAX**: SIZE_MAX is a macro constant in the C programming language that represents the maximum possible value of the size_t data type. It defines the absolute upper limit for the size of any object, array, or memory allocation that the hosting architecture can handle.

  Defined in

```c
#include <stdint.h>
```

7.1 Linkage

- **Linkage**: Linkage is about whether declarations of the same name refer to the same object/function across scopes or translation units.
  - **external linkage**: same entity can be referred to from other translation units
  - **internal linkage**: same entity only within this translation unit
  - **no linkage**: name is purely local; cannot be linked to another declaration

  C has mostly the same linkage / scope rules as C++, with one major difference. At file scope const variable have **external linkage**.

  C23 adds a C version of constexpr for objects. A WG14 proposal states that a file-scope constexpr object has static storage duration and internal linkage, meaning each translation unit gets a separate object. So in C23, file-scope constexpr also does not need static for header safety.

7.2 Double pointers

- **Double pointers**: In C, a double pointer is a pointer that stores the address of another pointer.

```c
int x = 10;

int *p = &x;    // p points to int
int **pp = &p;  // pp points to pointer-to-int
```

- **Modifying a pointer inside a function**: C passes arguments by value. If you pass a pointer to a function, the function receives a copy of that pointer.

  To modify the caller's pointer, pass the address of the pointer:

```c
void good_alloc(int **p) {
    *p = malloc(sizeof(int));
    **p = 42;
}

int main(void) {
    int *x = NULL;

    good_alloc(&x);

    printf("%d\n", *x); // 42

    free(x);
}
```

```c
size_t f(int** a) {
    size_t n_elements = 6;
    *a = malloc(sizeof(int) * n_elements);

    if (*a == NULL) return 0;

    int t = 0;
    for (size_t i = 0; i < n_elements; ++i) {
        (*a)[i] = t++;
    }
    return n_elements;
}

int* x;
size_t n = f(&x);
for (size_t i = 0; i<n; ++i) {
    printf("%d, ", x[i]);
}

free(x);
```

7.3 stdarg.h

- **Header**:

```
#include <stdarg.h>
```

This gives access to

```
va_list
va_start
va_arg
va_end
va_copy
```

- **Variadic functions**: A variadic function must have at least one named parameter before ....

```
void f(int x, ...) {...}
```

- **va_list**: va_list is an object that tracks the current position in the unnamed argument list.

You do not manually index into the stack or registers. The ABI decides where arguments live. Some may be in registers, some on the stack. va_list hides those details.

```
va_list args;
```

the arguments are split conceptually like this:

```
named/fixed parameters
unnamed variadic args
```

- **va_start**: Initializes args so it points to the first unnamed argument.

```
void f(int x, ...) {
    va_list args;

    // args points to first argument after x
    va_start(args, x);
}
```

The second argument must be the last named parameter before ....

- **va_arg**: Reads the next variadic argument as the type specified, then advances the cursor.

```
int value = va_arg(args, int)
```

This is dangerous because C does not store type information for variadic arguments.

- **va_end**: Cleans up the va_list.
- **Count parameter**: A variadic function has no built-in way to know how many extra arguments were passed. You must design a protocol.

```
int sum(int count, ...);
```

The first argument tells the function to read `count` values.

- **Sentinel value**: We can mark the end of the parameter pack by passing NULL as the last argument.

```
print_strings("hello", "world", "test", NULL);
```

- **Examples**:

```
int acc(size_t count, int x, ...) {
    int acc = x;

    va_list args;
    va_start(args, x);
    int arg = va_arg(args, int);

    for (size_t i = 0; i < count; ++i) {
        acc += arg;
        arg = va_arg(args, int);
    }

    va_end(args);

    return acc;
}
```

```
int acc2(size_t count, ...) {
    int acc = 0;

    va_list args;
    va_start(args, count);
    int arg = va_arg(args, int);

    for (int i = 0; i < count; ++i) {
        acc += arg;
        arg = va_arg(args, int);
    }

    va_end(args);

    return acc;
}
```

```
size_t ppack_len_count(const char* s1, va_list args) {
    if (s1 == NULL) return 0;
    size_t total_len = strlen(s1);

    const char* arg = NULL;
    while ((arg = va_arg(args, const char*)) != NULL) {
        size_t len = strlen(arg);
        // Rearranged total_len + len + 1 to avoid overflow
        if (total_len > SIZE_MAX - len - 1) {
            return 0;
        }
        total_len += len;
    }

    return total_len;
}
```

```c
char* str_concat(const char* s1, ...) {
    if (s1 == NULL) {
        return NULL;
    }

    va_list args;
    va_start(args, s1);

    size_t tlen = ppack_len_count(s1,args);

    va_end(args);

    char* final = malloc(tlen + 1);
    if (final == NULL) {
        return NULL;
    }

    // Moving write pointer. Final stays fixed to base
    char* mv_write = final;

    size_t len = strlen(s1);
    memcpy(mv_write, s1, len);
    mv_write += len;

    va_start(args, s1);

    const char* arg = NULL;
    while ((arg = va_arg(args, const char*)) != NULL) {
        len = strlen(arg);
        memcpy(mv_write, arg, len);
        mv_write += len;
    }

    va_end(args);

    *mv_write = '\0';

    return final;
}
```

7.4 C99

- **Designated initializers**: Designated initializers allow initializing struct fields by name.

```c
struct S {
    int x,y,z;
};

struct S s  = {.x = 10 };
struct S s2  = {.x = 10, .y = 20, .z = 30};

printf("%d, %d, %d\n", s.x, s.y, s.z); // 10, 0, 0
printf("%d, %d, %d\n", s2.x, s2.y, s2.z); // 10, 20, 30
```

- **Designate array indices**: You can also designate array indices:

```c
int arr[10] = {
    [5] = 6,
    [2] = 1
};

// 0, 0, 1, 0, 0, 6, 0, 0, 0, 0,
for (__auto_type i = 0; i < 10; ++i)
printf("%d, ", arr[i]);
```

- **___auto_type**: ___auto_type is a GNU C compiler extension supported by GCC and Clang that enables automatic type inference in the C programming language. It allows the compiler to automatically deduce the data type of a local variable from its initialization expression, providing functionality similar to the auto keyword in C++11

  - **Immediate Initialization Required:** You cannot declare a variable with ___auto_type without assigning a value immediately.
  - **Plain Identifiers Only:** The declarator must be a single, plain variable name. You cannot mix it with pointer or array syntax during declaration (e.g., ___auto_type *ptr = &x; is syntax error).
  - **Single Variable Limit:** You can only declare one variable per statement

  Before ___auto_type, type-generic macros relied on the typeof extension. However, typeof evaluates expressions with side effects twice. Using ___auto_type safely captures values exactly once, preventing double-evaluation bugs:

- **auto**: The behavior of the auto keyword in C depends entirely on the version of the C standard you are using. In traditional C (up to C17), auto is a storage class specifier used to declare variables with automatic storage duration, which means they are created when entering a block and destroyed upon exiting it. However, starting with the C23 standard, auto has been redefined to perform type inference, matching its modern behavior in C++

- **Compound literals**: Compound literals create unnamed objects inline.

```c
struct Point {
    int x;
    int y;
};

draw_point((struct Point){ .x = 10, .y = 20 });
```

They are also useful for arrays:

```c
int *p = (int[]){1, 2, 3, 4};
```

Inside a function, a compound literal has automatic storage duration. At file scope, it has static storage duration.

- **Variadic macros**: C99 added macros that accept a variable number of arguments.

```c
#define LOG(fmt, ...) \
fprintf(stderr, "[log] " fmt "\n", __VA_ARGS__)
```

- **String concatenation at compile time**: This is allowed because C concatenates adjacent string literals at compile time.

```c
printf("hello, " "world!");
```

Is translated as if you had written

```c
printf("Hello, world!");
```

The two string literals are not separate runtime strings. The compiler combines them during translation, before the program runs.

This rule exists mainly to make long strings easier to write:

```c
printf("This is a very long message "
"split across multiple source lines "
"but still one string.\n");
```

- **Inline functions**: C99 added the inline keyword.

```c
inline int square(int x) {
    return x * x;
}
```

The purpose is to allow function-like abstractions without the problems of macros.

```
#define SQUARE(x) ((x) * (x))
SQUARE(i++) // Increments i twice
```

An inline function would avoid the issue above.

C99 inline rules are slightly subtle, especially across translation units. For header-defined functions, many projects use static inline. This gives each translation unit its own internal-linkage copy.

```
static inline int square(int x) {
    return x*x;
}
```

- **Variable Length Arrays (VLAs)**: C99 added variable length arrays, or VLAs.

```
void f(int n) {
    int arr[n];

    for (int i = 0; i < n; ++i) {
        arr[i] = i;
    }
}
```

The array size is determined at runtime, but the object usually has automatic storage duration, typically stack-like storage.

VLAs are controversial. They are convenient, but they can be dangerous if n is large or untrusted:

C11 later made VLAs optional, so they are less universally portable than other C99 features.

- **Flexible array members**: C99 added flexible array members for structs with variable-sized trailing data.

```
struct Buffer {
    size_t len;
    unsigned char data[];
};
```

You allocate enough memory for the struct plus the extra array data:

```
struct Buffer *b = malloc(sizeof *b + 1024);

b->len = 1024;
b->data[0] = 42;
```

The flexible array member must be the last member of the struct.

- **__func__**: C99 added __func__, which expands to the current function name as a string.

```
void parse_expr(void) {
    printf("entered %s\n", __func__);
}
```

Memory concepts

- **uintptr_t, intptr_t**: uintptr_t and intptr_t are integer types from:

```
#include <stdint.h>
```

They are used to store pointer values as integers.

```
uintptr_t  // unsigned integer type capable of holding a
    ↪  pointer value
intptr_t   // signed integer type capable of holding a
    ↪  pointer value
```

if these types exist on your platform, converting a void * pointer to uintptr_t or intptr_t and then back to the same pointer type preserves the original pointer value.

```
int x = 42;

uintptr_t p_as_int = (uintptr_t)&x;

int *p = (int *)p_as_int;

printf("%d\n", *p); // 42
```

Pointers are not necessarily the same size as int, long, or long long. On some platforms, unsigned long might not be large enough to store a pointer. uintptr_t exists specifically for this purpose.

Usually, uintptr_t is the one you want. Pointer values are normally treated like raw addresses, and addresses are better represented as unsigned integers.

Use intptr_t only if you specifically need a signed integer representation of a pointer value.

- **Printing a pointer as an integer**: Normally, use %p for pointers

```
printf("%p\n", (void*)p);
```

But, if you want the numeric address

```c
#include <stdint.h>
#include <inttypes.h>

printf("%p\n", (void*)p);
printf("%" PRIuPTR "\n", (uintptr_t)p);
printf("%" PRIxPTR "\n", (uintptr_t)p);

// 0x7ffdceb0ff5c
// 140728071159644
// 7ffdceb0ff5c
```

On my machine, `PRIuPTR` expands to `lu`, and `PRIxPTR` expands to `lx`

Do not assume the format is %lu, because uintptr_t might not be unsigned long.

- **Alignment checks**:

```c
    if (((uintptr_t)p % 16) == 0) {
    // p is 16-byte aligned
}
```

```c
if (((uintptr_t)p & 15) == 0) {
    // p is 16-byte aligned
}
```

For an alignment A, where A is a power of two, the addr is aligned if

```c
addr % A == 0
addr & (A-1) == 0 // Equivalent
```

- **Aligning an address downward**: For power-of-two alignment, the formula is

```c
aligned = addr & ~(align - 1);
```

Consider the case where we want to round an address down to the previous 16-byte boundary. In that case, we know that the address is aligned to a 16-byte boundary if its bottom four bites are cleared, since $16 - 1 = 15 = 0xF = 0b1111$.

If we take an address and divide it by 16, then its remainder r must obey $0 \leqslant r < 16$. That is, the remainder is between zero and $16 - 1$ inclusive.

Consider a number a and an alignment A. In base two,

$$\begin{aligned} a =& \lambda_n 2^n + \lambda_{n-1} 2^{n-1} + \ldots + \lambda_\ell 2^\ell + \ldots \\ &+ \lambda_{\log_2(A)} 2^{\log_2(A)} + \lambda_{\log_2(A)-1} 2^{\log_2(A)-1} \\ &+ \lambda_{\log_2(A)-2} 2^{\log_2(A)-2} + \ldots + \lambda_1 2^1 + \lambda_0 2^0. \end{aligned}$$

Note that $\lambda_i \in \{0, 1\}$. Observe that

$$A = \lambda_{\log_2(A)} 2^{\log_2(A)}$$

when $\lambda_{\log_2(A)} = 1$. Further observe that all terms that come before this term are divisible by A. All terms that come before this term are in the form

$$\lambda_{\log_2(A)+k} 2^{\log_2(A)+k}.$$

for $k \in \mathbb{Z}_{>0}$, and $\log_2(A) + k \leqslant n$. Observe that if $\lambda_{\log_2(A)+k} = 1$,

$$\frac{2^{\log_2(A)+k}}{2^{\log_2(A)}} = 2^k \in \mathbb{Z},$$

and if $\lambda_{\log_2(A)+k} = 0$,

$$\frac{0}{2^{\log_2(A)}} = 0.$$

Thus, a will only have a remainder after being divided by A when all terms of the form

$$\lambda_{\log_2(A)-j} 2^{\log_2(A)-j}$$

have $\lambda_{\log_2(A)-j} = 0$, since none of these terms are divisible by A. Note that $j \in \mathbb{Z}_{>0}$ and $\log_2(A) - j \geqslant 0$

This is precisely why the check

```
a & (A-1) == 0
```

Checks if a has remainder zero when divided by A. Now, consider

```
~(A-1)
```

This is the number with all terms greater than or equal to $2^{\log_2(A)}$ non zero, and all terms less than $2^{\log_2(A)}$ zero. In essence, all bits left of and including the $(\log_2(A) + 1)$th one, and all bits to the right zero. For example, take $A = 16$, then

$$\sim (A - 1) = \sim (15) = 0b1111 \ldots 11110000.$$

So, the bitwise and of a number a with this mask turns off those bottom bits. Thus, it rounds a down to the nearest boundary such that it is A aligned.

- **Aligning an address upward**: For power-of-two alignment, the formula is

```
aligned = (addr + align - 1) & ~(align - 1);
```

Adding align - 1 moves the address such that it is inside the next highest boundary, then we take the bitwise and of the negation of the alignment -1 (see above). This rounds it down to the bottom of that next highest boundary. Thus, a round up of the original boundary.

- **Pointer tagging**: Some low-level code stores flags in unused low bits of aligned pointers.

```
#define FLAG 1u

uintptr_t tagged = ((uintptr_t)p) | FLAG;

int *real_ptr = (int *)(tagged & ~(uintptr_t)FLAG);
int flag_set = tagged & FLAG;
```

This only works when you know the pointer is aligned enough that the low bit is unused.

- **Dangers with uinptr_t**: This type does not make pointer arithmetic magically safe. The following is okay

```
uintptr_t n = (uintptr_t)p;
p = (int*)n;
```

This is much more dangerous

```
uintptr_t n = (uintptr_t)p;
n += 100;
p = (int *)n;
```

That may produce a pointer that does not point to a valid object. Dereferencing it would be undefined behavior unless you know exactly what memory layout you are working with.

- **Use calloc to allocate struct memory**: calloc is commonly used to allocate user-defined structures, especially when you want the allocated memory initialized to zero.

```
typedef struct Node {
    int value;
    struct Node *next;
} Node;

Node *node = calloc(1, sizeof(Node));

if (node == NULL) {
    /* allocation failed */
}
```

This allocates enough memory for one Node and sets all bytes to zero. In practice, this means:

```
node->value == 0;
node->next == NULL;   /* on normal hosted C implementations
   ↪  */
```

The C Abstract Machine

- **Idea**: The C abstract machine is the formal model that the C standard uses to define what a C program means.

  It is not a real CPU. It is an imaginary execution model with objects, values, expressions, side effects, sequence rules, storage durations, and undefined behavior. The compiler is allowed to generate any real machine code that behaves **as if** the program ran according to this abstract model

  That principle is called the as-if rule.

  When you write:

  ```c
  int x = 1;
  x = x+2;
  printf("%d\n", x);
  ```

  the C standard describes this as operations in the abstract machine:

  1. An int object x is created.
  2. The value 1 is stored in it.
  3. The expression $x + 2$ is evaluated.
  4. The result 3 is stored back into x.
  5. printf observes the value 3.

  The compiler does not have to literally generate instructions in that order. It only has to preserve the observable behavior of the program.

  So the compiler could optimize this to something morally like:

  ```c
  printf("%d\n", 3);
  ```

  because the observable result is the same.

- **Undefined behavior**: The abstract machine is also where undefined behavior gets its force.

  If your program does something the C standard says is undefined, then the abstract machine imposes no requirements on what happens.

- **Storage duration**: The abstract machine also classifies how long objects exist.

  ```c
  int global;              // static storage duration

  void f(void) {
      int local;           // automatic storage duration
      static int counter;  // static storage duration
      int *p = malloc(16); // allocated storage duration
  }
  ```

These categories matter because the abstract machine defines when objects begin and end their lifetimes.

```
int *bad(void) {
    int x = 10;
    return &x;
}
```

After bad returns, x no longer exists in the abstract machine. The returned pointer does not point to a live object anymore. Dereferencing it is undefined behavior.

- **Automatic storage duration**: In C programming, automatic storage duration refers to the lifetime of local variables. Memory is automatically allocated when the execution enters the block or function where the variable is defined, and it is deallocated as soon as the execution exits that block.

Idioms

- **(void*)-1**: Casting -1 to void* in C is a common idiom used to represent an error, sentinel, or special map value.

  While NULL (typically (void *)0) signifies "nothing" or "not found," APIs often need a distinct value to indicate an actual failure or error state.

  When you cast the signed integer -1 to a pointer type, the compiler converts its bit pattern into an address.

  On modern architectures utilizing two's complement arithmetic, -1 translates to all bits set to 1 (e.g., 0xFFFFFFFF on a 32-bit system or 0xFFFFFFFFFFFFFFFF on a 64-bit system).

  This effectively places the pointer at the very top of the addressable memory space, a region generally reserved for the operating system kernel. As a result, this pointer is safely outside the range of normal user-space application memory.

  Attempting to read or write to (void *)-1 (e.g., *(int *)ptr) will immediately trigger a segmentation fault or an access violation because user applications do not have permission to access that extreme address boundary.

  Converting integers directly to pointers is technically implementation-defined by the C standard. While it works predictably across standard compilers (GCC, Clang, MSVC), you should use the integer type uintptr_t or intptr_t from <stdint.h> if you are passing custom integer IDs inside pointer variables:

```
0  void* ptr = (void*)(uintptr_t)int_id;
```

- **brace-enclosed initializer list**: Consider

```
0  typedef struct _S {
1      int a,b,c;
2  } S;
```

  We can create an instance of type S and give values to its members with

```
0  S s = {1,2,3};
```

  If we omit some members in an aggregate initializer list, C initializes the omitted members as if they had static storage duration.

- **Zero-initializing**: $= \{0\}$ is a common C idiom for zero-initializing an aggregate, such as a struct, array, or union.

```c
typedef struct _S {
    int a;
    int* x;
} S;


int main(int argc, char** argv) {

    S s1;
    S s2 = {0};

    // Undefined behavior
    printf("%d\n", s1.a);

    // Definitely zero
    printf("%d\n", s2.a);

    // Not guaranteed to be NULL
    if (s1.x == NULL) {
        printf("NULL\n");
    } else printf("NOT NULL\n");

    // Definitely NULL
    if (s2.x == NULL) {
        printf("NULL\n");
    } else printf("NOT NULL\n");


    return EXIT_SUCCESS;
}
```

Memory allocator

- **sbrk and brk**:

```
0  int brk(void *addr);
1  void *sbrk(intptr_t increment);
```

brk() and sbrk() change the location of the program break, which defines the end of the process's data segment (i.e., the program break is the first location after the end of the uninitialized data segment). Increasing the program break has the effect of allocating memory to the process; decreasing the break deallocates memory.

brk() sets the end of the data segment to the value specified by addr, when that value is reasonable, the system has enough memory, and the process does not exceed its maximum data size

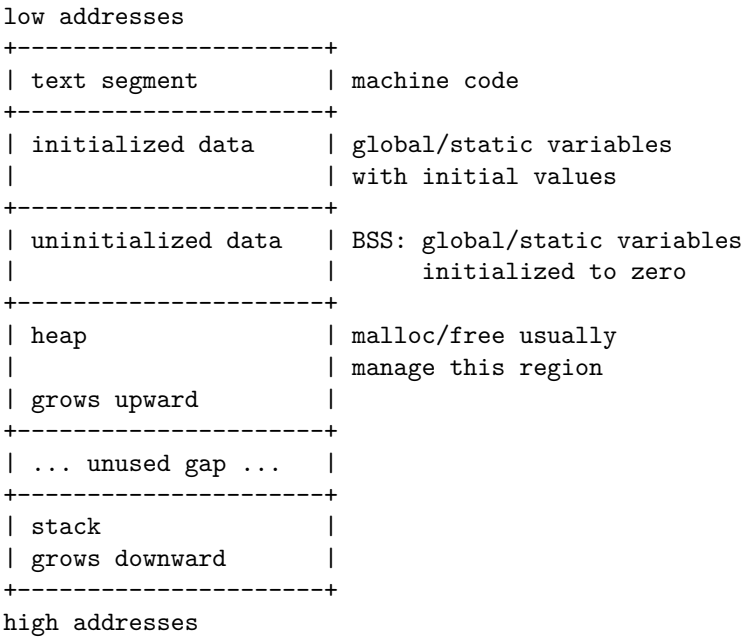
sbrk() increments the program's data space by increment bytes. Calling sbrk() with an increment of 0 can be used to find the current location of the program break.

On success, brk() returns zero. On error, -1 is returned, and errno is set to ENOMEM.

On success, sbrk() returns the previous program break. (If the break was increased, then this value is a pointer to the start of the newly allocated memory). On error, (void *) -1 is returned, and errno is set to ENOMEM.

sbrk is used by old/simple allocators to move the end of the process heap. It does not "allocate an object" directly. Instead, it asks the OS/kernel to make the process's data area larger, then the allocator manages that newly obtained region itself.

A traditional Unix process layout looks roughly like this:



The heap begins after the program's data segment, historically after the uninitialized data segment, also called BSS.

- **Program break**: The program break is the address marking the current end of the process's heap/data area.

```
heap start                                  program break
|                                                 |
v                                                 v
+---+-------------------------------------------+
|   | memory obtained from the OS for heap |
+---+-------------------------------------------+
```

- **System calls for obtaining memory**: A custom allocator needs to get large chunks of memory from the operating system.

  Historically, allocators use

```
sbrk()
brk
```

  Modern allocators often use:

```
mmap();
munmap();
```

- **Arenas**: An arena is a region of memory that an allocator manages as a unit.

  Suppose your allocator calls:

```
void *arena = sbrk(4096);
```

  Now you have a 4096-byte arena. Then a user asks for 32 bytes. You do not call sbrk(32) necessarily. Instead, you split part of the arena.

  So the arena is the larger pool, and each allocation is a block inside that pool. Calling the OS for memory is relatively expensive. An allocator usually does not want to ask the OS every time the user requests a tiny allocation.

  With sbrk, the traditional heap itself can be treated as one arena. When the arena runs out of usable free blocks, your allocator extends it.

So in a toy allocator, your "arena" may just be the continuous heap region between the first block you got from sbrk and the current program break.

- **mmap allocators**: mmap asks the OS for a separate virtual memory mapping. Calling it creates a new memory region unrelated to the sbrk heap.

```
.data / .bss
brk heap
...
[ mmap allocation A ]
...
[ mmap allocation B ]
...
stack
```

Each mapping can be independently destroyed with:

```
munmap(p,size);
```

That means an allocator can return an individual large allocation to the OS immediately, without needing it to be at the end of a contiguous heap.

With mmap, your allocator is managing independent memory mappings. With sbrk, your allocator is managing a single growing heap arena.

For example, if the user asks for 32 bytes, using mmap would be wasteful because mmap works at page granularity. The OS will usually map at least one page, commonly 4096 bytes. But if the user asks for 10 MB, using mmap is attractive because the allocator can later call munmap and return that whole region to the OS cleanly.

For mmap allocations, you still need metadata, but you must remember the exact mapping size, because munmap requires the address and length:

mmap is naturally page-oriented. If the page size is 4096, then an allocation using mmap generally rounds up to a multiple of the page size.

```
if (size >= MMAP_THRESHOLD) {
    allocate_with_mmap(size);
} else {
    allocate_from_heap_or_sbrk(size);
}
```

That is closer to how production allocators think: use internal arenas for normal allocations, and direct mappings for large blocks.

- **Process**:
  - OS gives allocator a large memory region.
  - Allocator splits that region into smaller blocks.
  - User calls malloc/free implementation that uses the allocator.
  - Allocator hands out and recycles blocks.
- **Block metadata**: Every allocated or free block usually has a hidden header.

```c
typedef struct block {
    size_t size;
    int free;
    struct block *next;
} block_t;
```

The real memory may look like this:

```
+----------------------+----------------------+
| allocator metadata   | user memory          |
| size, free, next     | returned by malloc   |
+----------------------+----------------------+
^                      ^
block header           pointer returned to user
```

If the user requests

```c
int *p = malloc(100);
```

Your allocator may actually reserve

```c
sizeof(block_t) + 100 bytes
```

Then it returns the address just after the metadata.

Note: free is a flag that records whether that block is currently available for reuse.

We call `free` an integer here because historically *C* did not have a boolean type. However, now that it does, you are free to mark `free` as a boolean.

The `size` member usually records the size of the usable memory area after the header.

```c
void *a = malloc(16);
void *b = malloc(64);
void *c = malloc(200);
```

```
+--------+-------------+
| header | 16 bytes    |
+--------+-------------+

+--------+-------------+
| header | 64 bytes    |
+--------+-------------+

+--------+-------------+
| header | 208 bytes   |
+--------+-------------+
```

If we call the size after the header the **payload**, then total block size is header plus payload.

```
sizeof(block_t) + block->size
```

- **alignof and max_align_t**: In C, alignof is an operator that returns the CPU-enforced memory alignment requirement of a type.

  Note that C11 introduced alignof as

  ```
  size_t _Alignof()
  ```

  with a macro

  ```
  #define _Alignof(x) alignof(x)
  ```

  found in

  ```
  #include <stdalign.h>
  ```

  While C23 made it available as `alignof` without the header `stdalign.h`.

  ```
  size_t alignof()
  ```

  Defined in <stddef.h>, max_align_t is a built-in type that guarantees an alignment boundary at least as large as any basic C data type (like double, long double, long long, or pointers).

  ```
  #include <stddef.h>
  ```

- **_Alignas / alignas**: The alignas specifier in C and C++ dictates the custom memory alignment for variables or user-defined types.

  Values must be an integral constant expression that evaluates to zero or a power of 2 (e.g., 1, 2, 4, 8, 16). If multiple alignas specifiers are applied, the compiler respects the largest value.

  In C, it was introduced as _Alignas (in C11). As of C23, it is available as the alignas keyword.

  ```
  alignas(8) int my_variable;
  ```

If the type naturally requires a larger alignment (e.g., a hardware type demanding 8 bytes), using alignas(4) is ignored.

- **Alignment**: malloc must return memory aligned enough for any normal object type. For example, this should be valid

```
long double *x = malloc(sizeof(long double));
```

So your allocator cannot return arbitrary byte addresses. It must return properly aligned addresses.

If the allocator uses 8-byte alignment, then every returned block size should be a multiple of 8. So, if a user requests 13 bytes, the allocator must return 16 bytes.

For a general-purpose malloc-style allocator, choose an alignment that is valid for any ordinary C object the user might store in the returned memory.

```
alignment = _Alignof(max_align_t)
```

A portable alignment helper would be

```
#define ALIGNMENT _Alignof(max_align_t)

static size_t align_up(size_t n) {
    return (n + ALIGNMENT - 1) & ~(ALIGNMENT - 1);
}
```

This assumes ALIGNMENT is a power of two, which it normally is for C object alignments.

- **Total alignment**: The hardware is optimized around reading/writing fixed-size chunks that start at natural boundaries.

Modern CPUs often allow loads from arbitrary byte addresses, especially x86-64. However, internally, the memory system is still organized around aligned units.

Also, *C* declares that accessing an object through a pointer that is not properly aligned for that type is undefined behavior.

We must ensure that not only the users request is aligned, but also the block header. Then, for an alignment A,

```
(alignup(sizeof(block_t)) + alignup(sizeof(request))) &
↪  (A-1) == 0
```

Note that if A divides `sizeof(block_t)` and A divides `sizeof(request)`, then A divides the sum.

Suppose A did not divide the size of the block, then the size of the block plus the requested space might not be properly aligned, even if we align up the requested space. So, we must round up both sizes.

- The hardware wants data at convenient addresses.
- The ABI defines how objects are laid out and passed.
- The compiler assumes pointer types are correctly aligned.
- The C language makes misaligned typed access undefined behavior.
- Your allocator must satisfy those rules so that returned memory can safely hold arbitrary objects.

So, alignment is important.

- **Over aligned objects**: In programming and computer science, an over-aligned object is a data structure or variable that requires a memory alignment boundary larger than the system's default or standard limit (known as max_align_t)
- **Free lists**: A simple allocator usually keeps a linked list of blocks. When malloc(64) is called, the allocator searches the list for a free block large enough.

  when `free(p)` is called, the allocator marks the block as free.

  - **malloc**:
    - search free list
    - find a big enough free block
    - mark it used
    - return pointer to user memory
  - **free**:
    - find block header
    - mark block free

  If the allocator cannot find a free block large enough to satisfy the request, it usually asks the OS for more memory and creates a new block from that memory.

  The important idea is that malloc does not create memory from nothing. It manages memory it already has, and when that is insufficient, it asks the operating system for a larger region.

- **Two models**:

  1. **One linked list of all blocks**: Whether free or in use. This is the simpler beginner allocator design.
  2. **Separate free list**: More realistic allocators often maintain a list containing only free blocks.

     In this model, when a block is allocated, it is removed from the free list.

     When a block is freed, it is inserted into the free list

But usually there is not also a normal "in-use list" for allocated blocks. The allocator does not need one for ordinary operation. When the user calls

```
free(ptr)
```

The allocator can find the block header directly

```
block_t* block = (block_t*)ptr-1;
```

- **Finding the block header from the user pointer**: This is a core trick. If malloc returns a pointer after the header

```
+---------+----------------+
| header  | user memory    |
+---------+----------------+
          ^
          returned pointer
```

Then free(ptr) must recover the header by moving backward:

```
block_t* block = (block_t* ptr) - 1;
```

This works if the returned pointer was created like this:

```
return block + 1
```

Thus,

```
void my_free(void *ptr) {
    block_t *block = (block_t *)ptr - 1;
    block->free = 1;
}
```

Recall that arithmetic on pointers moves the pointer using the size of the type pointed to.

Note: If we round up the block header size, then

```
return (void*)(ptr + 1)
```

and similar operations is no longer valid, since there is now padding between the end of the header and the usable space. Instead,

```
return (void*)((char*)ptr + BLOCK_ALIGNMENT)
```

- **Splitting blocks**: Suppose the allocator has this free block:

```
+----------------------+
| free block: 1024 bytes|
+----------------------+
```

  Then the user requests 100 bytes. A simple allocator could give the entire 1024-byte block to the user, but that wastes memory.

  Instead, it can split the block:

```
before:

[ free 1024 ]

after malloc(100):

[ used 100 ][ free 900-ish ]
```

  You need to understand how to split one block into two valid blocks with separate headers.

- **Coalescing adjacent free blocks**: When blocks are freed, memory can become fragmented:

```
[ used ][ free ][ free ][ used ]
```

  The two adjacent free blocks should often be merged:

```
[ used ][ larger free ][ used ]
```

  This is called **coalescing**.

  Without coalescing, your allocator may fail large allocations even though there is enough total free memory.

  Note: You usually coalesce when a block is freed.

  1. mark block as free
  2. try to merge with previous physical neighbor if free
  3. try to merge with next physical neighbor if free
  4. insert/adjust the resulting block in the free list

  Coalescing means merging physically adjacent free blocks, not merely adjacent nodes in the free list.

- **Types of coalescing**:

1. **Immediate coalescing**: Coalesce every time the user frees a block.
2. **Deferred coalescing**: Do not coalesce immediately. Instead, only coalesce later, usually when allocation fails.

- **Finding the next header**: While coalescing, we can perform pointer arithmetic on the current block to get the address of where the next block header would be. But, how do we know that there is truly a block header there?

  If we track the position of the program break, then we can check that a header exists at that location if it is less than the program break.

```
char* next_address = (char*)block + HEADER_ALIGN +
↪  block->size;
if (next_address >= program_break) {
    // no header exists at this location.
}
```

- **Finding the previous header**: This is much harder. You cannot generally compute it from the current block alone, because the previous block may have any size.

  An easy design choice is to store links in the Header struct for the previous and next Headers.

- **Fragmentation**: There are two major kinds.

  1. **Internal fragmentation**: The allocator gives more memory than requested.
  2. **External fragmentation**: There is enough total free memory, but it is split into small pieces.

- **Allocation strategies**: When searching the free list, the allocator needs a policy.
  - **First fit**: Use the first free block large enough
  - **Best fit**: Use the smallest free block large enough
  - **Worst fit**: Use the largest free block
  - **Next fit**: Continue searching from the previous search position

- **Boundary tags**: More advanced allocators store metadata at both the beginning and end of a block.

```
+--------+-------------+--------+
| header | user memory | footer |
+--------+-------------+--------+
```

  This makes it easier to find neighboring blocks and coalesce them efficiently.

  Without boundary tags, coalescing can require scanning the whole list.

- **Bins and segregated free lists**: A simple allocator may keep one global free list.

More advanced allocators keep separate lists for different block sizes:

  - 8-byte blocks -> list
  - 16-byte blocks -> list
  - 32-byte blocks -> list
  - 64-byte blocks -> list
  - large blocks -> list

These are often called bins, size classes, or segregated free lists. This improves performance because malloc(16) does not need to search through large unrelated blocks.

- **Thread safety**: A basic allocator is not thread-safe. If two threads call malloc at the same time, they may corrupt the allocator's internal structures.

  You need to eventually understand:

  1. mutexes
  2. atomic operations
  3. per-thread arenas
  4. lock contention
  5. reentrancy

- **Debugging allocator bugs**: Allocator bugs are often severe because they corrupt memory silently.

  You need to understand common failure cases:

  1. double free
  2. use after free
  3. buffer overflow
  4. freeing invalid pointers
  5. metadata corruption
  6. returning unaligned pointers
  7. overlapping allocations
  8. losing blocks from the free list

  You should also write internal consistency checks.

```
void heap_check(void);
```

  This can verify that block sizes, free-list links, and address ranges make sense.

- **Minimum concepts for a first allocator**

  1. void *, char *, and pointer arithmetic
  2. Process heap and sbrk()
  3. Block headers
  4. Alignment
  5. Linked-list free list

6. First-fit allocation
7. Finding the header from a user pointer
8. Splitting free blocks
9. Marking blocks free
10. Coalescing adjacent free blocks

Libraries

- **Static vs dynamic**:
  - A static library is copied into the final executable at link time.

    On Linux, static libraries usually have the extension:

    ```
    .a
    ```

    You create one from object files using ar:

    ```
    ar rcs mylib.a file1.o file2.o ...
    ```

    Then link it into a program:

    ```
    cc main.o -L. -lmylib -o bin
    ```

    Advantages:
    * Easier deployment: one executable contains the library code.
    * No runtime dependency on the .a file.
    * Can be simpler for small projects.

    Disadvantages:
    * Larger executable.
    * Updating the library requires relinking the program.
    * Multiple programs using the same library each get their own copy of the code.
  - A dynamic/shared library is loaded separately, usually at program startup or sometimes later at runtime.

    On Linux, shared libraries usually have the extension:

    ```
    .so
    ```

    You build one like this:

    ```
    gcc -fPIC -c mgrant.c -o mgrant.o
    gcc -fPIC -c mrel.c -o mrel.o
    gcc -fPIC -c free_list.c -o free_list.o
    gcc -fPIC -c block.c -o block.o

    gcc -shared -o libmayalloc.so mgrant.o mrel.o
    ↪ free_list.o block.o
    ```

Then, link a program against it

```
cc main.o -L. -lmayalloc -o main
```

But unlike a static library, the executable does not contain the full library code. It records that it needs libmayalloc.so.

At runtime, the dynamic linker must find it.

```
LD_LIBRARY_PATH=. ./main
```

Advantages:

* Smaller executable.
* Multiple programs can share one loaded copy of the library code.
* Library can sometimes be updated without rebuilding the program.
* Required for mechanisms like LD_PRELOAD.

Disadvantages:

* Runtime dependency: the .so must be installed or discoverable.
* Versioning can become more complicated.
* Slightly more moving parts during build and deployment.

On Linux, put libraries in a directory the linker already searches.

```
/usr/lib
/usr/local/lib
/lib
```

For your own manually installed library, prefer:

```
/usr/local/lib
```

Public headers usually go in:

```
/usr/local/include
```

Then you can compile with:

```
gcc main.c -lmylib
```

If using a shared library, refresh the runtime linker cache:

```
sudo ldconfig
```

So the usual layout is

```
/usr/local/include/may_alloc.h
/usr/local/lib/libmayalloc.a
/usr/local/lib/libmayalloc.so
```

- **Notes about static libraries**: When you compile a project into a static library, let the target be

```
lib/libprojname.a
```

This is convention.

Then, we can move the header and .a

```
sudo cp lib/libprojname.a /usr/local/lib
sudo cp include/projname.h /usr/local/include
```

Note that when we use the -l flag, we omit lib from the name (and the file extension .a), as these are both automatically added.

Also, for static libraries, order matters. Put the library after the object file:

```
gcc obj/main.o -L/usr/local/lib -lmay -o bin/main
```

For static libraries, the linker scans left-to-right. When it sees -lmay before obj/main.o, it has not yet seen the unresolved mgrant reference, so it does not pull mgrant from libmay.a.

Finding memory leaks

- **gcc AddressSanitizer (ASan)**: Passing

```
-fsanitize=address
```

to gcc enables AddressSanitizer, usually abbreviated ASan. It instruments your program so that, at runtime, it can detect memory errors such as:

```
heap-buffer-overflow
stack-buffer-overflow
use-after-free
use-after-return
double-free
invalid free
```

For example,

```
gcc -O0 -fsanitize=address main.c -o main
```

If your program writes past the end of an allocation:

```
int *p = malloc(sizeof(int) * 4);
p[4] = 10;   // invalid: valid indexes are 0..3
```

ASan will usually stop the program and print a stack trace showing where the invalid access happened.

- **gcc LeakSanitizer (LSan)**:

```
-fsanitize=leak
```

This enables LeakSanitizer, usually abbreviated LSan. It checks whether heap allocations were never freed before the program exited.

For example,

```
gcc -O0 -fsanitize=leak main.c -o main
```

Often you combine the two:

```
gcc -O0 -fsanitize=leak,address main.c -o main
```

```
-fno-omit-frame-pointer
```

This tells the compiler: Do not remove the frame pointer register from generated function calls.

The frame pointer helps debugging tools reconstruct the call stack accurately. Without it, optimized code may be faster, but stack traces can become less reliable or harder to read. With it, tools like AddressSanitizer, LeakSanitizer, Valgrind, gdb, and profilers can usually show cleaner backtraces.

```
gcc -g -O0 -fsanitize=address,leak -fno-omit-frame-pointer
↪ main.c -o main
```

This helps produce output like:

```
allocated from:
    #0 malloc
    #1 make_node at list.c:12
    #2 append_node at list.c:30
    #3 main at main.c:45
```